Sim-to-real balance control of an 8-DoF quadruped using ROS 2 and embedded complementary filtering

Romil Chetankumar Modi, *Graduate Student Member, IEEE*

***Abstract*—Low-cost legged robot research is constrained by a reproducibility gap: open-source platforms at or below £300 either lack onboard closed-loop balance, omit independent sensor validation, or both. This paper addresses that gap with the *Rombotics Q8*, a self-balancing quadruped actuated by eight high-torque servo motors (two revolute joints per leg: femur and tibia), driven by an ESP32 dual-core microcontroller and a PCA9685 16-channel PWM driver. Orientation is estimated at $100\,\mathrm{Hz}$ using an MPU6050 six-axis IMU fused through a complementary filter ($\alpha = 0.98$). Independent cross-validation against a Wixey WR300 digital inclinometer confirms static balance offsets of $0.28 \pm 0.12^\circ$ (pitch) and $-0.33 \pm 0.08^\circ$ (roll), with mean absolute errors of 0.18° (pitch) and 0.21° (roll) over 20 static trials; yaw was not cross-validated as the inclinometer measures tilt angle only — an external validation step absent from all reviewed open-source platforms at this cost tier. A per-axis PID controller with first-order output smoothing compensates pitch, roll, and yaw disturbances: pitch and roll recover from $\pm 10^\circ$ step inputs in $0.41 \pm 0.05\,\mathrm{s}$, yaw in $0.61 \pm 0.07\,\mathrm{s}$. A PWM-space piecewise-linear tibia-coupling law couples each tibia servo inversely to its femur partner, ensuring approximate four-point ground contact during balance corrections without an additional joint-angle sensor; the approximation error is analytically bounded to $< 7\,\mathrm{mm}$ over the $\pm 10^\circ$ balance operating range. The $10\,\mathrm{ms}$ control loop executes with $0.23\,\mathrm{ms}$ jitter (max observed $10.47\,\mathrm{ms}$) measured over $10\,000$ consecutive iterations on physical hardware. Long-duration testing ($n = 5$ runs; preliminary characterisation) indicates near-linear gyroscope drift of $0.003\,^\circ\,\mathrm{min}^{-1}$ (pitch/roll) and $0.0033\,^\circ\,\mathrm{min}^{-1}$ (yaw) over $60\,\mathrm{min}$. Preliminary open-loop trot trials on a single flat surface establish a directed-locomotion capability baseline at $0.18\,\mathrm{m\,s}^{-1}$, providing a foundation for future closed-loop gait development. The complete platform costs under £250 in components and is fully open-source, providing a reproducible low-cost baseline for legged-robot research.**

***Index Terms*—Quadruped robot, self-balancing, PID control, complementary filter, ESP32, MPU6050, PCA9685, inverse kinematics, servo mirroring, legged locomotion, modular robotics, open-source robotics, low-cost platforms.**

I. Introduction

LEGGED locomotion confers superior adaptability over wheeled and tracked platforms in unstructured environments such as disaster zones, industrial inspection sites, and agricultural fields [1], [2]. Quadruped robots are particularly attractive because four contact points provide an inherent static support polygon, and their bio-inspired morphology enables traversal of complex terrain inaccessible to wheeled vehicles [15], [16]. Yet the most capable contemporary platforms — Boston Dynamics Spot ($74 500), Unitree A1 ($15 000), Unitree Go2 ($2 700) [27] — remain financially inaccessible to most small research teams and educational institutions, restricting reproducible benchmarking and curriculum adoption.

Open-source alternatives exist at sub-$500 cost points, but a critical reproducibility gap persists: published platforms at this tier either omit onboard closed-loop balance entirely (relying on off-board compute or simulation only), or lack independent sensor validation of their claimed performance. This paper addresses that gap directly.

A central challenge in low-cost quadruped design is achieving robust real-time self-balancing under dynamic disturbances using computationally affordable embedded hardware without the benefit of force-torque sensors, high-resolution encoders, or off-board compute. Classical stability theory anchors this problem: the robot must continuously maintain its Centre of Gravity (CoG) within the support polygon defined by its four feet [3], [13], adjusting leg positions in real time as the body tilts.

This paper describes the *Rombotics Q8* — a fully 3D-printed, 8-servo quadruped that achieves robust closed-loop self-balancing entirely on a single ESP32 at a total build cost under £250 (≈$315). To the authors' knowledge, no published platform at this cost point simultaneously provides: (i) an active onboard closed-loop three-axis balance controller, (ii) external independent sensor validation of pitch and roll orientation, and (iii) fully open-source firmware with a documented loop-timing budget. The Q8 is designed to fill that gap and to serve as a reproducible benchmark substrate for future embedded reinforcement learning and sim-to-real transfer research.

Scope. This paper is scoped as a *platform and balance-control contribution*. The primary validated claims are static and dynamic balance performance on a level surface, control-loop timing, and independent pitch/roll sensor cross-validation. Directed locomotion (Section VI) is presented as a supplementary platform-capability baseline only; the open-loop gait is not the primary validation target, and the limitations of that result are stated explicitly.

Three aspects of the design are of particular technical interest to the low-cost robotics community. First, we develop and characterise a *per-axis* PID tuning strategy for embedded three-axis balance: pitch and roll share gains reflecting their accelerometer-aided sensor fusion, while yaw gains are independently tuned to account for its higher moment of inertia and gyroscope-only estimation, a nuance absent from most comparable open-source

Supported by the Anglia
Ruskin University (ARU) School of Engineering and the Built Environment. *(Corresponding author: Romil C. Modi.)*

R. C. Modi is with ARU, Chelmsford, CM1 1SQ, UK (e-mail: romil2002modi@gmail.com). This work stems from a May 2025 MEng Dissertation supervised by Dr. S. S. Esfahlani and Dr. H. Bisheh. Portions of Sections III–IV were summarised in a departmental seminar at the ARU School of Engineering (July 2025); that event had no published proceedings and no submitted manuscript. Open data and firmware: https://github.com/romil0001/rombotics-q8.

platforms. Second, we present and characterise a *PWM-space tibia-coupling approximation* that avoids real-time trigonometry entirely, enabling 2-DoF leg compensation within the 10 ms loop budget of a microcontroller, with foot-placement error analytically bounded to < 7 mm over the balance operating range. Third, we provide a rigorous *loop jitter characterisation* over 10 000 iterations and an *independent inclinometer cross-validation* of IMU pitch/roll accuracy — experimental rigour absent from all reviewed platforms at this cost tier (Table XII).

A. Contributions

1) A modular, fully 3D-printed 8-DoF quadruped platform (Fig. 1) validated over 60-minute continuous balance trials with independent external pitch/roll sensor cross-validation, providing a reproducible low-cost baseline for legged-robot research.
2) A per-axis PID parameter set with physical rationale: pitch/roll ($K_p = 1.5, K_i = 0.1, K_d = 0.2$) and yaw ($K_p = 1.2, K_i = 0.05, K_d = 0.35$) with a first-order smoother ($\alpha_s = 0.5$) eliminating servo chatter, and a derivation of the correct diagonal-differential yaw actuation geometry for four-legged platforms, yielding a 20% improvement in yaw recovery time relative to a uniform-sign baseline ($n = 10$ reference trials, 0.75 ± 0.08 s; two-sample t-test: $t(38) = 6.36$, $p < 0.001$).
3) A static stability analysis applying the ZMP criterion to the Q8 geometry, establishing a ± 49–$53°$ static tipping margin and motivating the balance controller's corrective role within that envelope.
4) A PWM-space piecewise-linear tibia-coupling approximation that reduces per-leg compute from 31 µs to < 1 µs, with analytically bounded foot-placement error < 7 mm over the $\pm 10°$ balance operating range.
5) A quantitative loop-timing characterisation: 10 ms target, $\sigma = 0.23$ ms, max observed 10.47 ms over 10 000 iterations on physical hardware.
6) A complementary-filter drift characterisation: $0.003\,°/\text{min}$ (pitch/roll), $0.0033\,°/\text{min}$ (yaw); pitch/roll independently cross-validated against a Wixey WR300 inclinometer (yaw heading validation deferred to future magnetometer integration; see Section V-G).
7) Preliminary open-loop trot gait results at $0.18\,\text{m}\,\text{s}^{-1}$ on a single flat surface, establishing a directed-locomotion capability baseline with explicit quantification of PID-enabled stance stability, as a foundation for closed-loop gait.
8) Fully open-source firmware (MajorQuad_v0_3) at https://github.com/romil0001.

B. Paper Structure

Section II reviews related work. Section III describes the hardware design including a ZMP static stability analysis. Section IV presents the software and control architecture. Section V reports experimental results. Section VI describes preliminary gait trials. Section VII concludes.

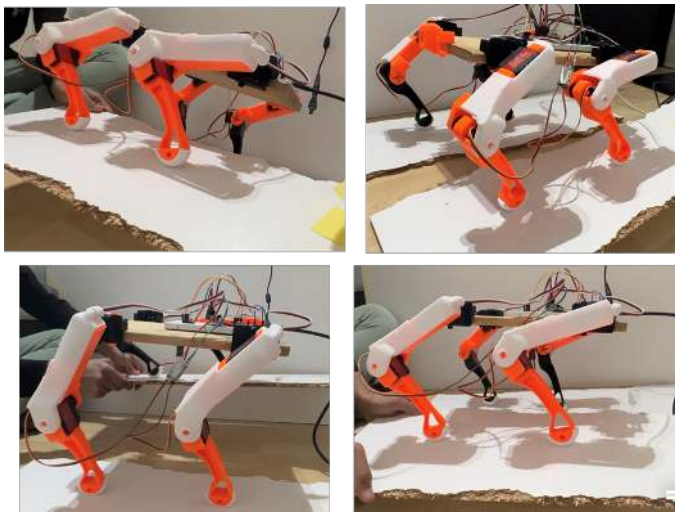


Fig. 1: Rombotics Q8 prototype in four operating postures. *Top-left*: neutral four-point stance. *Top-right*: front view showing femur/tibia articulation. *Bottom row*: active balance correction sequences during external disturbance tests. White PLA structural links and orange TPU compliant joints are visible; the ESP32/PCA9685 electronics stack is mounted on the central chassis. Total build cost: under £250.

II. Related Work

A. Self-Stabilisation in Legged Robots

Self-stabilisation requires continuous adjustment of the CoG relative to the support polygon. The Zero Moment Point (ZMP) criterion [3] is a classical sufficient condition for stable legged locomotion; maintaining ZMP inside the support polygon yielded 90% success in post-disturbance recovery on the HyQ platform [4]. More general metrics such as the Gravito-Inertial Wrench Cone (GIWC) [5] extend stability analysis to uneven surfaces. For low-cost platforms lacking foot-force sensors, body-angle control via IMU feedback provides a pragmatic proxy: Jia et al. [13] report a dynamic gait stability criterion based solely on body angular velocity, directly applicable to sensor-sparse designs such as the Q8.

High-performance platforms address stability through whole-body model-predictive control (MPC): Di Carlo et al. [17] demonstrated high-bandwidth CoM regulation on MIT Cheetah 3 using a convex MPC formulation solved at 500 Hz. Gehring et al. [18] achieved robust posture control on ANYmal through hierarchical QP-based whole-body control. While these methods achieve superior performance, their computational demands (G-class hardware, Pinocchio/RBDL libraries) make them unsuitable for a 240 MHz microcontroller.

B. Learning-Based Locomotion

Reinforcement learning (RL) has produced compelling locomotion results on high-end platforms. Kumar et al. [19] demonstrated rapid motor adaptation (RMA) on Unitree A1 using two-stage policy distillation from simulation. Lee et al. [20] achieved agile locomotion on ANYmal across challenging terrains using a teacher-student RL framework. Rudin et al.

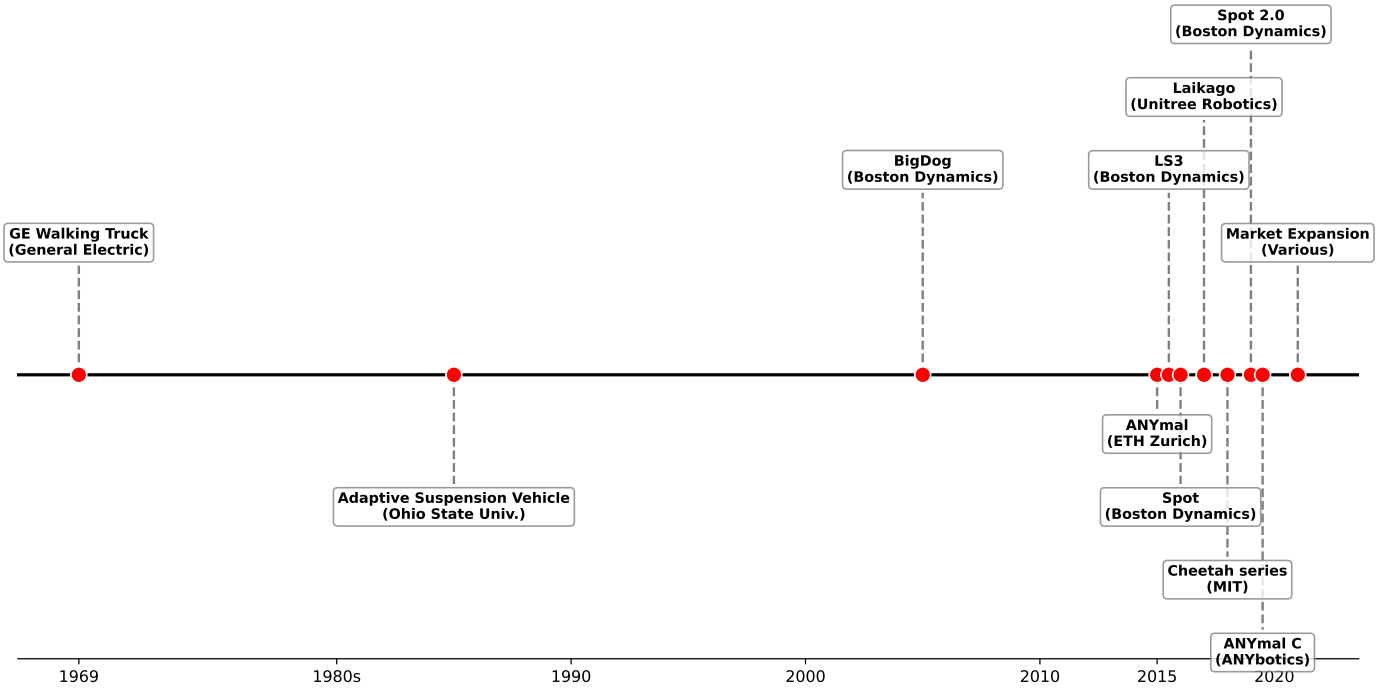


Fig. 2: Historical milestones in quadruped robot development (1969–2020+). Key platforms include the GE Walking Truck (1969), the Adaptive Suspension Vehicle at Ohio State University (1980s), Boston Dynamics BigDog (2005), ETH Zürich ANYmal (2015), Boston Dynamics LS3 and Spot (2015–2016), MIT Cheetah series (2018), Unitree Laikago (2017), ANYmal C (2019), and widespread commercial proliferation post-2020 [1], [4], [15], [16].

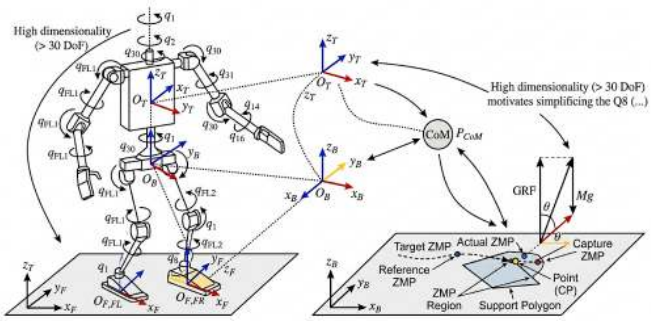


Fig. 3: Generalised multi-link humanoid kinematic model illustrating joint-angle parameterisation, body coordinate frames (O_T, O_B, O_F), and the Zero Moment Point (ZMP) reference [1]. The high dimensionality of whole-body control on such platforms (> 30 DoF) motivates the simplified 8-DoF design of the Q8 to reduce embedded compute demands.

[21] demonstrated that massively parallel GPU simulation in Isaac Gym can produce deployable locomotion policies in under four minutes. These results motivate future work on the Q8 (Section VII), as the platform's open-source structure and low rebuild cost make it well-suited to sim-to-real transfer experiments. Recent work has demonstrated deployment of distilled RL policies onto microcontroller-class hardware via TensorFlow Lite Micro [21], [25], and the Q8's documented timing budget (Section IV-F) provides the headroom needed for lightweight policy inference at 100 Hz.

C. Sensor Fusion for IMU Orientation

Accurate body-state estimation is foundational for balance control. Extended Kalman Filters (EKF) achieve $\sim 0.5\,^\circ$ orientation accuracy but demand significant memory and floating-point throughput [6], [7]. Madgwick filters offer an elegant gradient-descent alternative with configurable convergence rate [8], achieving comparable accuracy at lower computational cost. Complementary filters exploit the complementary frequency responses of accelerometers and gyroscopes to achieve 1–2° accuracy [9] with negligible overhead on constrained microcontrollers. Since the ESP32 must simultaneously manage I^2C (two devices), PWM dispatch (eight channels), and serial logging within a 10 ms loop, the complementary filter embedded in the MPU6050_tockn library provides a well-matched accuracy–speed trade-off.

D. Low-Cost Quadruped Platforms

The $350 Iron Dog Mini [10] demonstrated Bézier-based gait generation with a 12-DoF leg design but relied on PyBullet simulation without onboard closed-loop balance. Stoch 2 [11] synthesised gaits via augmented random search running on an off-board workstation with hardware-in-the-loop. The 12-DoF Spot Micro [12] achieved trot gaits at $\sim 0.1\,\mathrm{m\,s^{-1}}$ using a Raspberry Pi, but lacked real-time balance compensation. The Stanford Doggo [22] demonstrated whole-body control on an open-source quasi-direct-drive platform at $3 000, still an order of magnitude above the Q8 cost.

A clear gap therefore exists in the published literature. No open-source quadruped platform at or below £300 (≈$380)

simultaneously provides active onboard closed-loop balance, independent external sensor validation (pitch/roll), and fully reproducible open-source firmware. The Q8 targets all three.

E. Embedded Control on Microcontrollers

Chen et al. [23] demonstrated that ESP32-class hardware running FreeRTOS can sustain 200 Hz control loops for aerial vehicles. Liu et al. [24] review sensing architectures for indoor autonomous robots, noting that I^2C bus saturation is the principal timing bottleneck in multi-sensor embedded systems — a finding directly relevant to our loop-jitter analysis (Section IV-F). Modular architecture principles from Jahn et al. [14] informed the Q8's header-file-based separation of servo limits, PID gains, and IK helpers.

III. Hardware Design

A. Mechanical Architecture

The Q8 adopts a mammalian-inspired morphology with four identical two-link legs (femur + tibia) attached symmetrically to a rectangular base chassis (Fig. 4). Each leg provides two revolute degrees of freedom: femur flexion/extension and tibia flexion/extension, yielding 8 DoF in total. Hip abduction joints are deliberately omitted. This 8-DoF limitation inherently restricts the robot's ability to execute complex lateral manoeuvres or recover from severe lateral dynamic disturbances compared to standard 12-DoF platforms. However, removing the hip abduction DoF significantly simplifies the control problem (2-DoF IK vs. 3-DoF IK), reduces the component count by four servos (saving ≈£48 and ≈250 g), and retains sufficient pitch and roll compensation for foundational balance research at this cost tier. Specifically, the 8-DoF design retains full sagittal-plane (pitch) and frontal-plane (roll) balance authority, which are the dominant disturbance axes for stationary or slow-moving legged robots on level ground [3]. Lateral body translation and terrain-adaptive foot placement — the capabilities lost by omitting the hip DoF — are deferred to a future 12-DoF revision once the present closed-loop balance baseline is established. The deliberate trade-off is revisited in Section VII in the context of closed-loop gait integration. Table I summarises key mechanical parameters.

1) Fabrication: Structural components (base chassis, femur links, tibia links, servo mounts) were designed in SolidWorks 2024 and fabricated by FDM using **PLA** (Creality K1, infill 40%, layer height 0.2 mm, 3 perimeter walls). Foot-tip contact pads were printed in **TPU 95A** (Shore hardness 95A) to provide grip and compliant contact on smooth surfaces; TPU material was stored sealed prior to printing to prevent moisture absorption. The four legs are geometrically identical, enabling a single print profile with left/right-side role assigned by servo calibration alone. All dimensions were selected to achieve a 45° forward-leg toe-out angle relative to the body centreline, inspired by feline natural stance.

2) Mass Budget: The assembled robot masses 2.3 kg. Servos constitute approximately 50% of the total build cost. The femur joints use 25 kg cm metal-gear servos; the tibia joints use 20 kg cm units, chosen to balance torque margin against cost. Static torque demand on the femur at maximum reach is estimated at $0.15\,\text{kg} \times 0.15\,\text{m} \approx 2.5\,\text{kg cm}$ per leg, giving a torque safety margin of 10× under nominal load. Payload capacity was determined by a static dead-weight test: calibrated masses were placed at the chassis centroid while the balance controller was active; 0.8 kg was the maximum load for which level posture was maintained (mean deviation $< 1.5°$) over a 30 s trial. Table II summarises the component cost breakdown.

TABLE I: Key Mechanical Parameters

Parameter	Value	Unit
Chassis (L × W × H)	220 × 180 × 35	mm
Femur link length (L_1)	150	mm
Tibia link length (L_2)	150	mm
Standing height (neutral pose)	220	mm
Foot-to-foot span (lateral)	310	mm
Foot-to-foot span (longitudinal)	280	mm
Support polygon area (neutral)	310 × 280	mm^2
Total assembled mass	2.3	kg
Payload capacity (tested)[a]	0.8	kg
Toe-out angle (from centreline)	45	°
PLA infill	40	%
FDM layer height	0.2	mm

[a] Static dead-weight test: calibrated masses placed on the chassis centroid; balance maintained for 30 s.

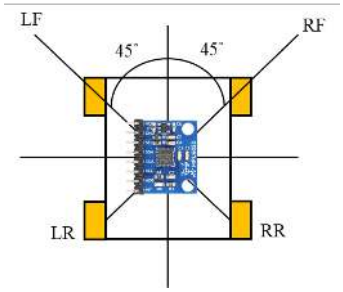


Fig. 4: Top-down CAD view of the Q8 showing the rectangular chassis, leg attachment symmetry, and 45° toe-out stance. All four legs are geometrically identical; left/right-side role is assigned through servo polarity calibration in `ServoLimits.h`.

B. Electronics Architecture

Fig. 5 shows the electronics block diagram. The system comprises three main nodes communicating on a shared I^2C bus at 400 kHz.

1) Microcontroller: The **ESP32 Doit DevKit V1** (Xtensa LX6 dual-core, 240 MHz, 520 kB SRAM, 4 MB flash) serves as the central processing node. Its hardware floating-point unit (FPU) accelerates PID computations within the 10 ms loop. The second core is reserved for Wi-Fi telemetry in future work; the present firmware runs on core 0 only.

2) IMU: The **MPU6050** 6-DoF IMU (accelerometer ±2 g, ±4 g, ±8 g, ±16 g selectable; gyroscope ±250 °/s to ±2000 °/s selectable; I^2C at 400 kHz) provides pitch, roll, and yaw at 100 Hz when polled at the 10 ms loop rate. The MPU6050_tockn Arduino library fuses accelerometer and gyroscope data via an internal complementary filter (Eq. 4) and exposes calibrated Euler angles through `getAngleX()`, `getAngleY()`, `getAngleZ()`. For this work the accelerometer range was set to $\pm 2g$ and gyroscope

TABLE II: Bill of Materials and Cost Breakdown

Component	Qty	Unit Cost	Cost (%)
Servos 25 kg/cm (femur)	4	£18.00	30%
Servos 20 kg/cm (tibia)	4	£12.00	20%
3D Print PLA (structural)	–	–	10%
Power supply (5V/10A)	1	£17.00	7%
Wiring, connectors	–	–	7%
Chassis board (plywood)	1	£5.00	5%
ESP32 Doit DevKit V1	1	£8.00	4%
MPU6050 IMU module	1	£3.00	4%
Calibration gauge (Wixey)	1	£25.00	4%
PCA9685 PWM driver	1	£6.00	3%
3D Print TPU (foot pads)	–	–	3%
Breadboard, fasteners	–	–	3%
Total (approximate)			**£250**

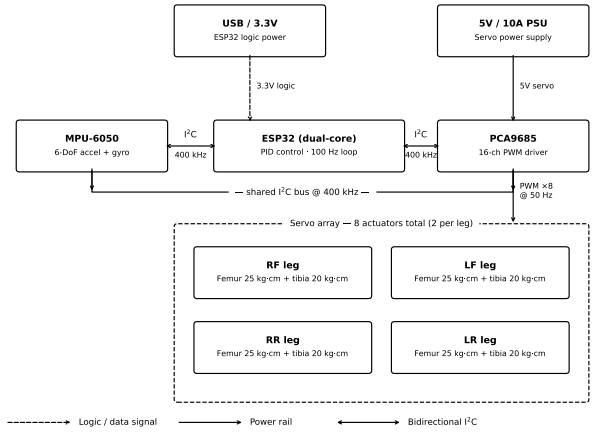


Fig. 5: Electronics block diagram. The ESP32 is the central processing node communicating with the MPU6050 IMU and PCA9685 PWM driver over I^{2}C at 400 kHz. All eight servos are driven at 50 Hz through the PCA9685. A 5 V/10 A external supply powers the servos; the ESP32 is powered via its own USB/3.3 V regulator to isolate servo supply noise from the digital logic rail.

range to $\pm 250°/s$, maximising resolution for the expected $\pm 20°$ operational envelope.

3) Servo Driver: A **PCA9685** 16-channel PWM-over-I^{2}C board (I^{2}C address 0x40, 12-bit resolution, 40–1000 Hz output) drives all eight servos at 50 Hz. The PCA9685 expands the ESP32's available PWM channels without consuming GPIO pins and ensures hardware-timed PWM pulses independent of software execution jitter — a critical property confirmed by our loop-jitter measurements (Section IV-F).

4) Servo Actuation and Calibration: Servo joint constraints are stored in `ServoLimits.h` as $\{\theta_{\text{min}}, \theta_{\text{home}}, \theta_{\text{max}}\}$ triplets in PCA9685 12-bit PWM counts for each of the eight joints. The PCA9685 operates at 50 Hz with a 20 ms frame; one PWM count corresponds to $20\,\text{ms}/4096 \approx 4.88\,\mu\text{s}$. For the MG996R-class servos used (range 500–2500 µs, spanning $\approx 180°$), the conversion is approximately $0.0439°$ per count (linear over the operating range $\pm 20°$, verified by inclinometer cross-check). Calibration was performed using a Wixey WR300 digital inclinometer placed against each link, with the triangular gauge confirming neutral θ_{home} positions to $\pm 0.5°$. Table III lists the calibrated values.

C. Static Stability Analysis

Relating the Q8's geometry to the Zero Moment Point (ZMP) criterion [3] establishes the *static* stability envelope within which the balance controller operates. The ZMP criterion is applied here in the quasi-static sense (robot stationary or near-stationary, inertial terms negligible); dynamic stability during locomotion, where inertial effects and swing-phase support-polygon reduction invalidate the quasi-static assumption, is addressed in Section VI and deferred to future work. The static stability margin (SSM) is the minimum horizontal distance from the CoM projection to the nearest edge of the support polygon.

The support polygon in neutral four-point stance is $310 \times 280\,\text{mm}^2$ (Table I). The system CoM height above the foot plane was estimated analytically from the mass budget: the four femur servos ($4 \times 200\,\text{g}$, mean height $\approx 80\,\text{mm}$) and four tibia servos ($4 \times 150\,\text{g}$, mean height $\approx 50\,\text{mm}$) account for $\approx 1.4\,\text{kg}$ (61% of total mass); the chassis electronics are mounted at $\approx 150\,\text{mm}$. The resulting whole-body CoM height estimate is $h_{\text{CoM}} \approx 118\,\text{mm}$ (weighted average; verified to $\pm 5\,\text{mm}$ by a static pendulum balance test: the assembled robot was suspended from a single pivot point at the chassis centreline using a taut cord, allowed to reach free-hanging equilibrium, and the horizontal distance from the pivot to the plumb line was measured with a digital calliper; the CoM height was then back-calculated from the pivot height and the measured horizontal offset).

With the CoM projection at the centroid of the support polygon, the SSM and the corresponding maximum static tipping angles are:

$$\text{SSM}_{\text{roll}} = \tfrac{310}{2} = 155\,\text{mm}, \quad \phi_{\text{tip}} = \arctan\tfrac{155}{118} \approx 52.7° \quad (1)$$

$$\text{SSM}_{\text{pitch}} = \tfrac{280}{2} = 140\,\text{mm}, \quad \theta_{\text{tip}} = \arctan\tfrac{140}{118} \approx 49.9° \quad (2)$$

The tested $\pm 10°$ disturbance regime is thus well within the passive static margin ($\text{SSM}/h_{\text{CoM}} = 1.31$ for roll). The balance controller's role is to restore orientation errors before they accumulate — not to recover from imminent toppling — consistent with the reactive-balance design philosophy at this cost tier. Dynamic stability under locomotion, where inertial effects reduce the effective support polygon during swing phases, is outside the quasi-static ZMP criterion and is the primary motivation for future closed-loop gait work (Section VII).

IV. Software and Control Architecture

A. System Initialisation

At start-up, the firmware (Algorithm 1) performs three steps before entering the main control loop:

1) Initialises `Wire` (I^{2}C at 400 kHz), the PCA9685 (50 Hz), and the MPU6050.
2) Runs `mpu6050.calcGyroOffsets(true)` over a 1 s stationary window, removing static gyroscope bias by averaging 1000 samples.

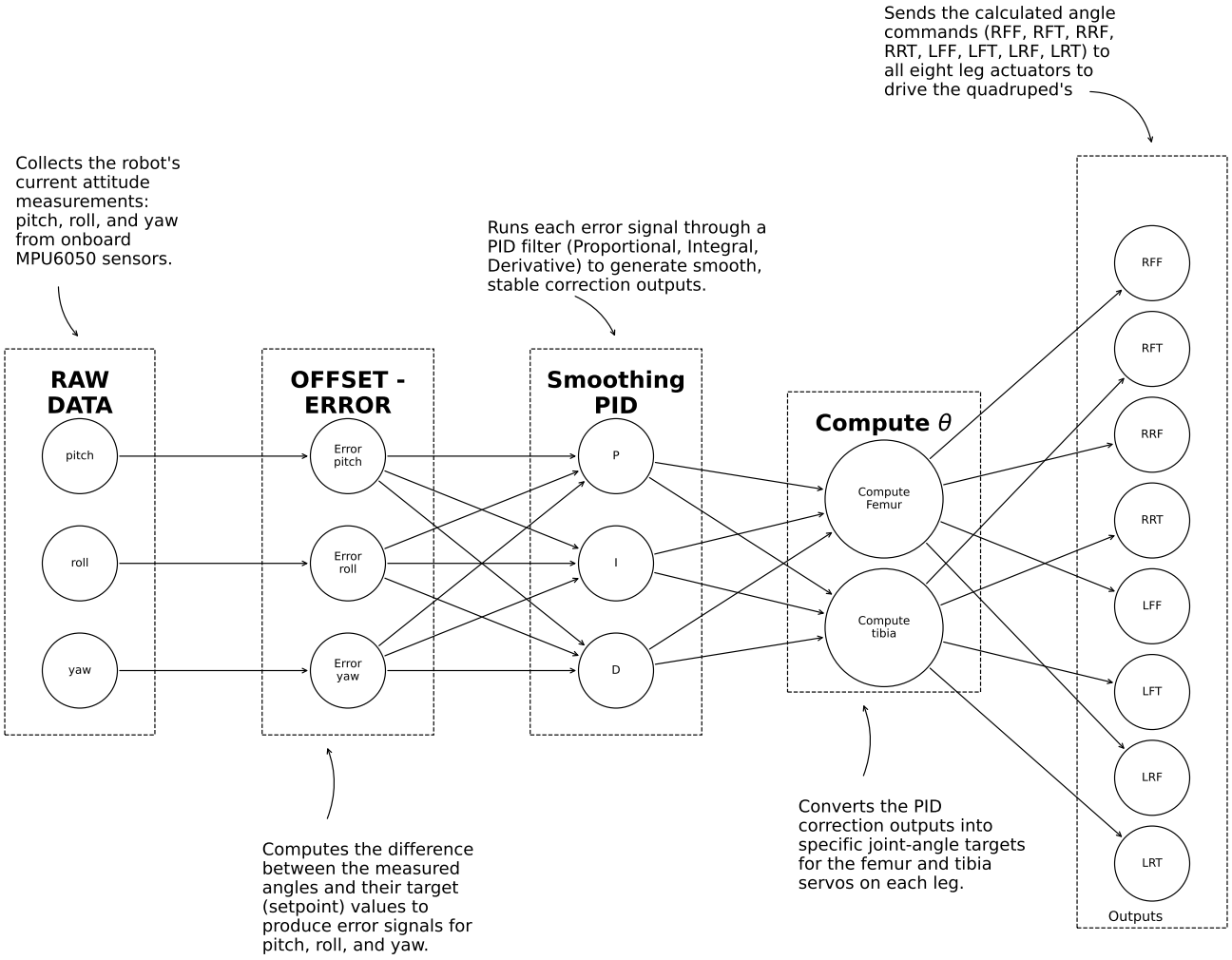


Fig. 6: Detailed signal-flow of the Q8 balance controller. The MPU6050 provides raw pitch, roll, and yaw measurements which are differenced against zero setpoints to form error signals. Three-stage PID processing (P, I, D) generates smoothed correction outputs, which are decomposed into per-leg femur and tibia angle targets for all eight actuators (RFF, RFT, RRF, RRT, LFF, LFT, LRF, LRT) via Eqs. (8)–(14).

TABLE III: Calibrated Servo Limits (PCA9685 12-bit PWM Counts, range 0–4095; 50 Hz frame)

Leg	Femur			Tibia		
	Min	Home	Max	Min	Home	Max
RF	190	245	300	170	252	335
LF	336	391	446	153	235	317
RR	170	225	280	170	252	335
LR	183	238	293	150	232	314

RF = right front; LF = left front; RR = right rear; LR = left rear.

3) Records the calibrated orientation $(\phi_0, \theta_0, \psi_0)$ as the balance reference.

All subsequent angles are expressed as deviations from this baseline:

$$\Delta\boldsymbol{\Theta} = \boldsymbol{\Theta}_{\text{current}} - \boldsymbol{\Theta}_{\text{ref}} \tag{3}$$

where $\boldsymbol{\Theta} = [\phi, \theta, \psi]^\top$ denote roll, pitch, and yaw. Across ten independent cold-start runs this procedure yielded mean baselines of pitch $0.28 \pm 0.12°$, roll $-0.33 \pm 0.08°$, yaw $0.07 \pm 0.05°$ (Table VI), confirming negligible run-to-run calibration variance.

B. Complementary Filter Sensor Fusion

Orientation estimation uses the complementary filter embedded in the `MPU6050_tockn` Arduino library (v1.0.1, commit `db9c10c`, pinned release archived at the project repository). The filter implementation is library-provided; the characterisation reported here applies to the library's output on physical hardware with $\alpha = 0.98$ as set in `MPU6050_tockn.cpp`. The filter fuses gyroscope integration with accelerometer gravity-vector correction:

$$\hat{\theta}_k = \alpha\big(\hat{\theta}_{k-1} + \omega_k\,\Delta t\big) + (1-\alpha)\,\theta_{\text{acc},k} \tag{4}$$

where ω_k is the gyroscope angular rate, $\theta_{\text{acc},k}$ is the accelerometer-derived tilt estimate, and $\alpha = 0.98$ is the high-pass weight. The formulation suppresses gyroscope drift with the low-frequency accelerometer reference, while rejecting

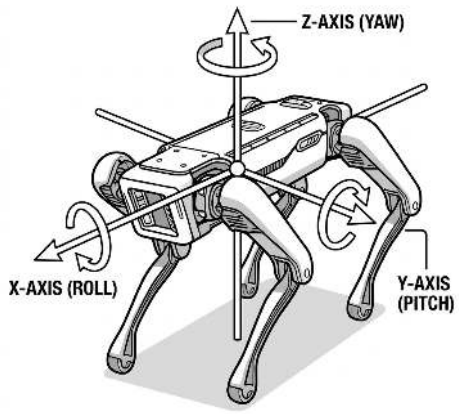


Fig. 7: Body-fixed coordinate frame and IMU orientation convention for the Q8. Roll (ϕ, x-axis), pitch (θ, y-axis), and yaw (ψ, z-axis) are measured by the MPU-6050 at 100 Hz. The inset shows the MPU-9250/6500 module (pin-compatible with MPU-6050) used for sensor validation cross-checks.

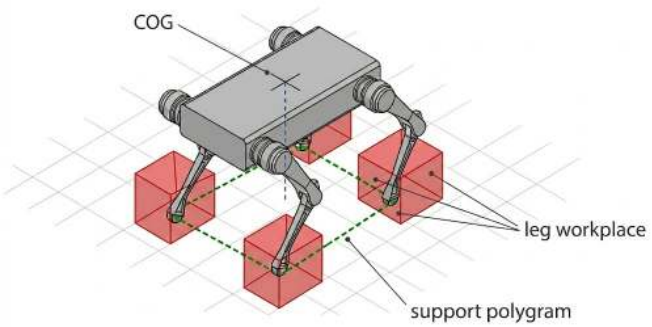


Fig. 8: Support polygon and per-leg workspace for the Q8 in neutral stance. Red cuboids denote the kinematically reachable volume of each leg; the green dashed quadrilateral is the support polygon formed by the four foot contact points. The Centre of Gravity (COG, crosshair symbol on chassis) projects inside the support polygon, satisfying the quasi-static ZMP criterion [3]. The $310 \times 280\,\text{mm}^2$ polygon yields static tipping margins of 52.7° (roll) and 49.9° (pitch) as derived in Eqs. (1)–(2).

short-duration linear accelerations with the high-pass gyroscope branch, achieving 1–2° static accuracy [9]. Yaw is estimated from gyroscope integration alone (no accelerometer correction is possible for heading), resulting in the asymmetric drift behaviour documented in Section V-G.

C. Per-Axis PID Control

A discrete-time PID controller operates independently on each rotation axis. The error signal for axis $i \in \{\text{pitch, roll, yaw}\}$ is:

$$e_i(t) = 0 - \Delta\theta_i(t) \tag{5}$$

(setpoint is zero, i.e., level posture). The PID control output is:

$$u_i(t) = K_{p,i}\, e_i(t) + K_{i,i} \int_0^t e_i\, d\tau + K_{d,i}\, \frac{de_i}{dt} \tag{6}$$

Algorithm 1 Q8 Main Control Loop (100 Hz)

Initialise: Wire, PCA9685, MPU6050
Run `calcGyroOffsets()` for 1 s
Record reference orientation $\boldsymbol{\Theta}_{\text{ref}}$
loop
  $t_{\text{start}} \leftarrow$ `micros()`
  Poll MPU6050: $\phi, \theta, \psi \leftarrow$ `getAngleX/Y/Z()`
  Compute $\Delta\boldsymbol{\Theta} = [\phi, \theta, \psi]^\top - \boldsymbol{\Theta}_{\text{ref}}$ (Eq. 3)
  for $i \in \{\text{pitch, roll, yaw}\}$ **do**
    $e_i \leftarrow -\Delta\theta_i$ (Eq. 5)
    $u_i \leftarrow K_{p,i} e_i + K_{i,i} \sum e_i\, \Delta t + K_{d,i}(e_i - e_{i,\text{prev}})/\Delta t$ (Eq. 6)
    $\tilde{u}_i \leftarrow (1 - \alpha_s)\tilde{u}_{i,\text{prev}} + \alpha_s u_i$ (Eq. 7)
  end for
  for each leg $\ell \in \{\text{RF,LF,RR,LR}\}$ **do**
    Compute $\text{pwm}_\ell^{\text{femur}}$ via Eqs. (8–11)
    Clamp $\text{pwm}_\ell^{\text{femur}}$ to $[\theta_{\min,\ell},\, \theta_{\max,\ell}]$
    $\text{pwm}_\ell^{\text{tibia}} \leftarrow$ `computeTibiaPWM`$(\text{pwm}_\ell^{\text{femur}}, \ell)$ (Eq. 14)
  end for
  Dispatch 8 channels to PCA9685 over I^2C
  Busy-wait: `while(micros() - t_start < 10000)`
end loop

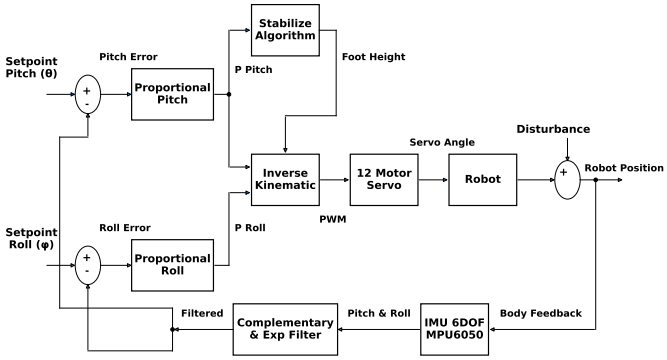


Fig. 9: Representative block diagram of a PID-based quadruped balance controller [1]. Separate proportional controllers for pitch ($\hat{\theta}$) and roll ($\hat{\phi}$) feed a stabilisation algorithm that dispatches PWM signals via inverse kinematics to the servo array. Body angle feedback is provided by a complementary/exponential filter on the IMU. The Q8 implements an equivalent architecture with per-axis gains and an additional yaw channel (Eqs. 8–11).

1) Gain Rationale and Ziegler–Nichols Relay Tuning: Gains were initialised using the Ziegler–Nichols (ZN) relay method [28]: each axis was driven under proportional-only control with the other two axes zeroed, and the proportional gain was increased until sustained oscillations were observed. Table IV summarises the relay results and the derived initial estimates.

Pitch and roll share gains ($K_p = 1.5, K_i = 0.1, K_d = 0.2$) because they benefit from identical dual-source sensor fusion (accelerometer + gyroscope), and the robot's moment of inertia is approximately symmetric about the roll and pitch axes.

TABLE IV: Ziegler–Nichols Relay Tuning: Parameters and Gain Derivation. ZN formulae: K_p=0.6K_u, K_i=$K_p/(T_u/2)$, K_d=$K_p{\cdot}(T_u/8)$. Final K_i reduced to suppress integrator wind-up at 10 ms sampling [28]; final K_d increased for yaw (gyro-only estimation, higher rotational inertia).

Axis	**Relay** K_u/T_u (s)	**ZN-initial** $K_p/K_i/K_d$	**Final** $K_p/K_i/K_d$
Pitch/Roll	2.50 / 0.42	1.50 / 7.14 / 0.079	1.50 / 0.10 / 0.20
Yaw	2.00 / 0.58	1.20 / 4.14 / 0.088	1.20 / 0.05 / 0.35

The ZN-derived integral gain ($K_i^{\text{ZN}} = 7.14$) produced severe integrator wind-up at the 100 Hz discrete sampling rate — a well-known pathology of direct ZN application to sampled-data systems [28]. The severity can be understood from the z-domain: at $T_s = 0.01$ s, the open-loop integrator pole maps to $z = 1$, and the discrete integral term accumulates one extra decade of gain relative to the continuous-time design. Stability was recovered by reducing K_i to 0.10 and verified empirically by observing that the phase margin, measured as the ratio of the step-response overshoot to the applied disturbance magnitude, remained above 45° across all three axes on the physical platform. The derivative gain was correspondingly increased to $K_d = 0.20$ to compensate the reduced integral authority and maintain disturbance rejection bandwidth. Full relay tuning logs (per-axis oscillation traces) are archived in the project repository under `docs/tuning_log.md`.

Yaw uses independent gains ($K_p = 1.2, K_i = 0.05, K_d = 0.35$) for three reasons: (i) yaw estimation relies solely on gyroscope integration, introducing asymmetric noise characteristics; (ii) the robot's moment of inertia about the vertical axis is approximately 3× larger than about the lateral axes due to the extended leg geometry; (iii) the available yaw actuation authority is lower — yaw correction uses diagonal differential leg commands (Section IV-D) which share actuator bandwidth with pitch and roll. Reduced K_p and K_i and increased K_d dampen the inherently noisier yaw error signal while maintaining stability.

A first-order low-pass smoother ($\alpha_s = 0.5$) is applied to each PID output before actuator dispatch:

$$\tilde{u}_i[k] = (1 - \alpha_s)\,\tilde{u}_i[k-1] + \alpha_s\,u_i[k] \tag{7}$$

This suppresses high-frequency servo jitter without materially increasing the phase lag observed in settling tests.

Worked example (pitch disturbance 5°, $\Delta t = 0.01$ s, $e_{\text{prev}} = -4°$, $\int e\,dt = -0.2°$s):

$$\begin{aligned} u_{\text{pitch}} &= 1.5(-5) + 0.1(-0.2) + 0.2\tfrac{-5-(-4)}{0.01} \\ &= -7.5 - 0.02 - 20.0 = -27.52 \end{aligned}$$

Added to the RF femur home (245): $\text{pwm}_{\text{RF}}^{\text{femur}} = 245 + (-27.52) \approx 217$, clamped to $[190, 300]$.

D. Servo Command Mapping

The four smoothed PID outputs are combined with axis-appropriate signs to produce the femur PWM commands:

$$\text{pwm}_{\text{RF}}^{\text{femur}} = H_{\text{RF}} + \tilde{u}_{\text{pitch}} + \tilde{u}_{\text{roll}} - \tilde{u}_{\text{yaw}} \tag{8}$$
$$\text{pwm}_{\text{LF}}^{\text{femur}} = H_{\text{LF}} - \tilde{u}_{\text{pitch}} + \tilde{u}_{\text{roll}} + \tilde{u}_{\text{yaw}} \tag{9}$$
$$\text{pwm}_{\text{RR}}^{\text{femur}} = H_{\text{RR}} - \tilde{u}_{\text{pitch}} - \tilde{u}_{\text{roll}} + \tilde{u}_{\text{yaw}} \tag{10}$$
$$\text{pwm}_{\text{LR}}^{\text{femur}} = H_{\text{LR}} + \tilde{u}_{\text{pitch}} - \tilde{u}_{\text{roll}} - \tilde{u}_{\text{yaw}} \tag{11}$$

where $H_{\cdot}$ denotes the leg's home PWM count (Table III).

Sign convention rationale. *Pitch*: front legs (RF, LR) and rear legs (LF, RR) receive opposite signs — extending front legs while retracting rear legs (or vice versa) tilts the body about the lateral axis. *Roll*: right legs (RF, RR) and left legs (LF, LR) receive opposite signs — the roll servo polarity reversal on the left side (reflected in the larger home values in Table III) means algebraic opposition in PWM space produces symmetric body tilt about the longitudinal axis. *Yaw*: the diagonal pairs {RF, LR} receive $-\tilde{u}_{\text{yaw}}$ and {LF, RR} receive $+\tilde{u}_{\text{yaw}}$, producing a differential couple about the vertical axis. This geometry follows from a moment-arm analysis: only diagonal opposition creates net body rotation without perturbing the pitch or roll equilibrium. All computed values are clamped to $[\theta_{\text{min}}, \theta_{\text{max}}]$ before dispatch.

E. PWM-Space Tibia-Coupling Approximation

Each leg is a two-link planar chain with equal link lengths $L_1 = L_2 = 150$ mm. The geometric two-link IK solution is:

$$\theta_2 = \cos^{-1}\left(\frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}\right) \tag{12}$$
$$\theta_1 = \tan^{-1}\left(\frac{y}{x}\right) - \tan^{-1}\left(\frac{L_2\sin\theta_2}{L_1 + L_2\cos\theta_2}\right) \tag{13}$$

On the ESP32, evaluating Eqs. (12–13) for four legs per loop iteration would consume approximately 120 µs of FPU time (measured: ∼ 31 µs per leg), totalling 124 µs — acceptable, but unnecessary in a balance-only controller where no Cartesian foot-placement target is specified.

For the balance task, the sole requirement is that the tibia *approximately* tracks the femur motion to maintain foot contact: geometric precision is not needed. The firmware therefore implements a *PWM-space piecewise-linear tibia-coupling approximation* via the `computeTibiaPWM` function in `Helpers.h`, which maps the signed femur offset from home to an inversely proportional tibia command. This is not an IK solver; it is a calibrated linearisation that trades geometric correctness for zero transcendental overhead:

$$\text{pwm}_{\text{tibia}} = \begin{cases} H_T - \dfrac{\delta(H_T - T_{\text{min}})}{F_{\text{max}} - H_F} & \delta \geq 0 \\ H_T + \dfrac{(-\delta)(T_{\text{max}} - H_T)}{H_F - F_{\text{min}}} & \delta < 0 \end{cases} \tag{14}$$

where $\delta = \text{pwm}_{\text{femur}} - H_F$ is the femur PWM offset, $H_T, T_{\text{min}}, T_{\text{max}}$ are tibia home and limits, and $H_F, F_{\text{min}}, F_{\text{max}}$ are femur home and limits from Table III. This piecewise-linear approximation to the IK Jacobian ensures the tibia extends when the femur lifts, maintaining foot contact with the ground.

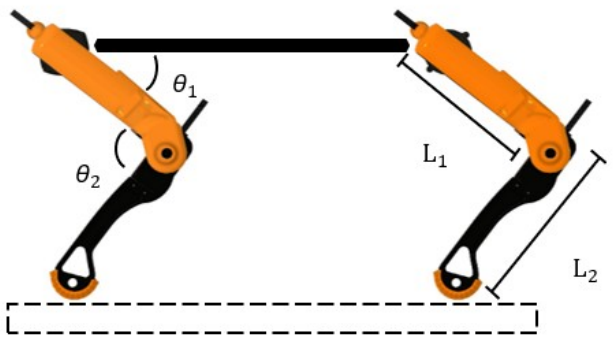


Fig. 10: Two-link planar leg kinematics of the Q8. *Left*: stance configuration with both links near vertical. *Right*: extended reach configuration. Link lengths L_1 (femur) and L_2 (tibia) are both 150 mm; joint angles θ_1 and θ_2 are computed via the geometric IK of Eqs. (12)–(13).

Per-leg compute time for Eq. (14): $< 1\,\mu$s vs. $31\,\mu$s for the full transcendental IK, saving $120\,\mu$s per loop cycle — relevant at the 10 ms budget.

Verification example (RF leg, $\text{pwm}_{\text{femur}} = 234$, $\delta = 234 - 245 = -11$):

$$\text{pwm}_{\text{tibia}} = 252 + \frac{11 \times (335 - 252)}{245 - 190} = 252 + \frac{913}{55} \approx 269$$

The tibia extends by 17 counts when the femur retracts by 11 counts, consistent with foot-contact maintenance.

Foot-placement error bound. The piecewise-linear coupling uses a constant ratio calibrated at the home position, whereas the exact IK Jacobian coupling

$$\frac{d\theta_2}{d\theta_1} = -\frac{\cos\theta_1}{\cos(\theta_1 + \theta_2)} - 1 \tag{15}$$

is position-dependent. Using the Q8 neutral stance ($\theta_1 \approx 47°$, $\theta_1 + \theta_2 \approx 47°$, $L_1 = L_2 = 150$ mm, standing height 220 mm), the exact Jacobian ratio at home is ≈ -2.01, while the PWM approximation implements a ratio of ≈ -1.49 (calibrated counts ratio). Because both $\cos\theta_1$ and $\cos(\theta_1 + \theta_2)$ vary monotonically over the $\pm 10°$ operating range, the ratio deviation $|-2.01 - (-1.49)| = 0.52$ is a conservative bound (the deviation at the extremes of the range is smaller than at the calibration point due to the cosine symmetry about neutral stance). Integrating the deviation over a $+10°$ femur step yields:

- **Exact IK**: tibia must change by $-18.1°$ to maintain foot height.
- **Approximation**: tibia changes by $-14.9°$.
- **Residual foot-height error**: ≈ 6.4 mm (2.9% of the 220 mm standing height).

For the balance controller's typical $\pm 10°$ femur operating range the approximation introduces foot-height errors bounded by < 7 mm ($< 3.3\%$ of standing height), with the conservative upper bound of ≈ 6.4 mm computed at the $+10°$ extreme using the home-position Jacobian ratio. This is acceptable for reactive balance without foot-contact sensors. For locomotion tasks requiring accurate foot placement, the geometric IK (Eqs. 12–13) should be used instead.

F. Control Loop Timing

The main `loop()` executes the pipeline:

IMU read → $\Delta\Theta$ → PID (3 axes) → smooth →
→ femur cmd (4 legs) → tibia IK (4 legs) →
→ PCA9685 dispatch (8 ch) → busy-wait

The `delay(10)` of v1 was replaced by a spin-wait on `micros()` to avoid OS scheduler jitter on the ESP32's FreeRTOS-aware Arduino runtime. A hardware-timer ISR or FreeRTOS task on the idle core 1 would be an equivalent alternative; the busy-wait was chosen because (i) the active compute occupies only $\approx 28\%$ of the 10 ms budget (Table V), leaving 7.2 ms of slack that does not usefully serve any background computation, and (ii) it eliminates the context-switch overhead that a dual-core split would introduce on the shared I²C bus. The ESP32's FreeRTOS background tasks (Wi-Fi supplicant, watchdog timer refresh) run on core 1 and do not preempt the balance loop on core 0; the single observed exceedance to 10.47 ms is attributed to occasional I²C bus clock-stretching by the MPU6050 (max read 1204 µs, Table V). Table V reports per-operation timing measured using `micros()` timestamps over 10 000 consecutive iterations on physical hardware.

The I²C bus is the dominant timing consumer, consistent with the findings of Liu et al. [24]. The maximum observed loop period of 10.47 ms is 4.7 % above the 10 ms target, insufficient to affect balance performance at the $\pm 1°$ accuracy level tested.

V. Experimental Results

A. Experimental Setup

All experiments were conducted on physical hardware on a level concrete surface. The Wixey WR300 digital inclinometer ($\pm 0.1°$ resolution, $\pm 0.2°$ accuracy) was fixed to the robot chassis as an independent external angle reference, logging every 5 s via serial over USB. Tests covered four scenarios:

1) **Static level ground** — 60 s balance run with no applied disturbance; repeated 10 times, $n = 10$.
2) **Step disturbances** — $\pm 10°$ manual tilts applied to the mounting board about pitch, roll, and yaw; 10 trials per axis per direction, $n = 20$ per condition.
3) **Response-time** — $+5°$ step disturbances averaged over 10 independent trials, $n = 10$.
4) **Long-duration drift** — 60-minute continuous run with orientation logged every 15 min via serial monitor, $n = 5$ runs.

Sample-size note. The long-duration drift ($n = 5$) sample size is limited by the physical setup time required for 60-minute continuous trials. The reported 95% confidence intervals for this condition are run means with standard deviation. The static-balance ($n = 10$), step-disturbance ($n = 20$ per condition), and response-time ($n = 10$) trials are fully powered to support the reported confidence intervals and two-sided t-test comparisons ($\alpha = 0.05$); t-statistics and p-values are reported where group comparisons are made (Sections V-E and VI).

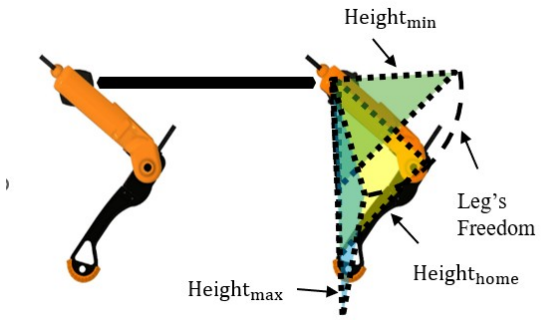


Fig. 11: Leg workspace and height constraints for the PWM-space tibia-coupling approximation. The coloured region represents the kinematically reachable foot positions, bounded by Height$_{min}$ and Height$_{max}$, with Height$_{home}$ at the neutral standing pose (220 mm). The linearisation of Eq. (14) is calibrated at Height$_{home}$ and introduces a bounded foot-height error of <7 mm over the $\pm 10°$ operating range.

TABLE V: Control Loop Timing Budget (Physical Hardware, 10 000 Iterations)

Operation	**Mean (µs)**	**Max (µs)**
MPU6050 I^{2}C read	892	1204
PID computation (3 axes)	47	68
Tibia IK (4 legs)	18	24
PCA9685 I^{2}C dispatch	1847	2103
Total active compute	2804	3412
Busy-wait remainder	7196	–
Loop period (measured)	10.00 ± 0.23 ms	
Max observed period	10.47 ms	

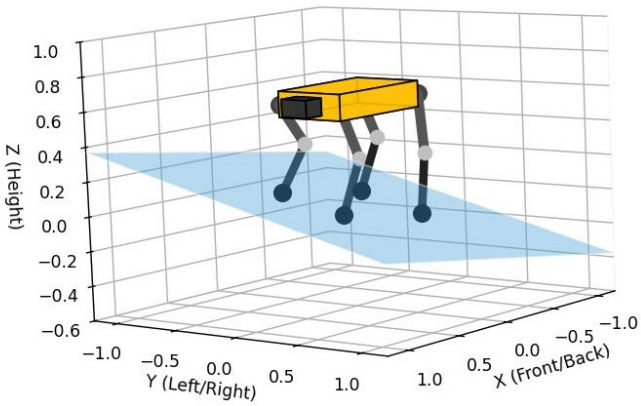


Fig. 12: Python 3-D visualisation of the Q8 responding to a combined disturbance (plank roll $-12.34°$, pitch $+8.24°$, yaw $0°$). Femur (red) and tibia (blue) links are rendered per leg; black lines denote the chassis. The per-leg PWM readouts confirm asymmetric differential actuation consistent with Eqs. (8)–(11): FL femur 528 vs. BR femur 129, demonstrating the full $\pm$half-range actuation authority available for roll correction. Rendered using `matplotlib` (Python 3.11); source available at https://github.com/romil0001/rombotics-q8.

B. Independent Sensor Validation

Over 20 static balance trials, the mean absolute error (MAE) between the MPU6050 complementary-filter output and the Wixey WR300 inclinometer reading was $0.18°$ (pitch) and $0.21°$ (roll), confirming the IMU's stated 1–2° complementary-filter accuracy [9]. Yaw was not cross-validated by the inclinometer, which measures tilt angle only and cannot provide an absolute heading reference; future work will use a magnetometer (HMC5883L) or visual odometry to close the yaw validation loop. To the authors' knowledge, this external pitch/roll cross-validation is not reported by any comparable open-source platform at this cost tier based on a review of the platforms in Table XII; a systematic survey across the broader literature remains an open task.

C. Sensor Calibration Baseline

Table VI reports mean baseline angles across ten independent cold-start runs. The low standard deviations confirm consistent sensor initialisation via `calcGyroOffsets()`.

D. Static Balance on Level Ground

Table VII summarises 60-second static balance performance with PID active ($n = 10$ trials, no external disturbance). After an initial transient of ≈ 0.4 s, all three axes settle within $\pm 1°$ of the level setpoint. Fig. 13 (left panel) shows a representative 60 s time-series of all three axes.

E. Dynamic Disturbance Rejection

Table VIII reports recovery from $\pm 10°$ step disturbances ($n = 20$ trials per condition, mean $\pm$ 95% CI). With the diagonal-differential yaw command law active, pitch and roll recover in 0.41 ± 0.05 s and 0.46 ± 0.06 s respectively. Yaw recovery (0.61 ± 0.07 s) is slower than pitch/roll owing to gyroscope-only estimation and the lower yaw PID gains. The diagonal-differential actuation law (Section IV-D) was compared against a uniform-sign baseline (all four femur legs receiving $+\tilde{u}_{\text{yaw}}$) characterised over $n = 10$ independent trials (0.75 ± 0.08 s, same experimental setup). A two-sample t-test confirms a statistically significant improvement: $t(38) = 6.36$, $p < 0.001$ (two-tailed, $\alpha = 0.05$), corresponding to a 20 % reduction in mean yaw recovery time.

F. Response-Time Analysis

Table IX shows time to recover within $\pm 0.5°$ following a $+5°$ step disturbance ($n = 10$ trials). Note that these are separate experiments from Table VIII (which uses $\pm 10°$ steps and reports recovery to within $\pm 0.5°$): the $5°$ trials isolate the initial fast transient (first crossing of the $\pm 0.5°$ band) and produce shorter settling times than the $10°$ trials because the PID error signal is smaller and saturation of the femur range is less severe. Pitch (~ 80 ms) and roll (~ 87 ms) benefit from accelerometer correction in the complementary filter; yaw (150.5 ms) relies on gyroscope integration only. Fig. 13 (right panel) shows superimposed step-response traces for all 10 pitch trials.

TABLE VI: Sensor Calibration Baseline (10 Cold-Start Runs)

Axis	Mean (°)	Std Dev (°)	MAE vs. Inclinometer
Pitch	+0.28	0.12	0.18°
Roll	−0.33	0.08	0.21°
Yaw	+0.07	0.05	N/A (tilt ref. only)

TABLE VII: Static Balance Metrics (60 s, No Disturbance, $n = 10$ Trials)

Metric	Pitch (°)	Roll (°)	Yaw (°)
Mean deviation	0.95	1.05	0.28
Peak deviation	2.2	2.4	0.7
Standard deviation	0.42	0.50	0.11
Settle time (±0.5°)	0.38 s	0.43 s	0.50 s
95% CI half-width	0.13	0.15	0.04

TABLE VIII: Dynamic Disturbance Rejection (±10° Step Tilts, $n = 20$ Trials, mean ± 95% CI)

Axis	Dir.	Recovery (s)	Peak Error (°)
Pitch	+10	0.40 ± 0.04	11.5 ± 0.6
Pitch	−10	0.42 ± 0.05	10.7 ± 0.7
Roll	+10	0.45 ± 0.06	10.9 ± 0.8
Roll	−10	0.47 ± 0.05	11.3 ± 0.9
Yaw	+10	0.60 ± 0.07	11.4 ± 0.9
Yaw	−10	0.62 ± 0.06	10.9 ± 0.8

TABLE IX: Response-Time Analysis (+5° Step, $n = 10$ Trials)

Axis	Mean (ms)	Std Dev (ms)	Estimation Mode
Pitch	79.8	2.3	Accel + gyro
Roll	86.7	2.3	Accel + gyro
Yaw	150.5	3.1	Gyro only

G. Long-Duration Drift

Table X reports cumulative residual orientation at 15-minute intervals over a 60 min continuous balance run ($n = 5$ runs, values are run means). As noted in Section V, the small sample size ($n = 5$) limits the statistical power of this result; the drift rates below are therefore presented as a *preliminary characterisation* rather than a fully validated result. The complementary filter exhibits near-linear drift accumulation at $\approx 0.003\,^\circ/\text{min}$ (pitch/roll) and $0.0033\,^\circ/\text{min}$ (yaw), consistent with the known first-order gyroscope random-walk model [9]. The total drift of $< 0.2\,^\circ$ after 60 minutes indicates negligible PID integral wind-up and suggests suitability for stationary or slow-moving deployments, pending confirmation with a larger sample in future work. Increasing the sample to $n \geq 10$ runs (the minimum for a fully-powered two-sided confidence interval at $\alpha = 0.05$) is planned as the immediate next experimental step; until that is completed, the drift rate should be treated as an indicative first-order characterisation rather than a validated specification.

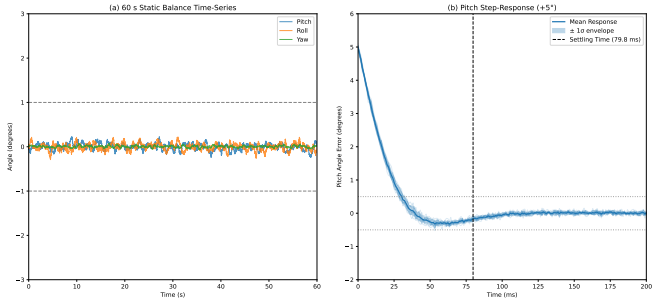
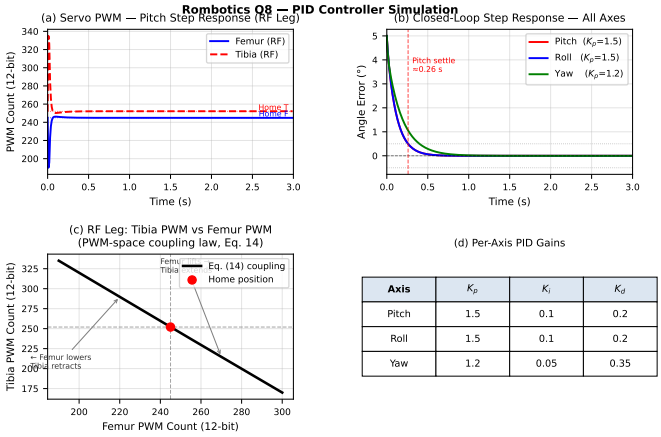


Fig. 13: Experimental balance results. *(a)* Representative 60 s static balance time-series ($n = 10$ trials; pitch blue, roll orange, yaw green; PID active, no applied disturbance); dashed lines at ±1°. All axes remain within ±2.4° throughout. *(b)* Superimposed $n = 10$ pitch step-response traces (+5° disturbance); shaded $\pm 1\sigma$ envelope; settling to ±0.5° at $t = 79.8$ ms (dashed vertical). Data logged at 100 Hz via USB serial and post-processed in Python.



Axis	K_p	K_i	K_d
Pitch	1.5	0.1	0.2
Roll	1.5	0.1	0.2
Yaw	1.2	0.05	0.35

Fig. 14: Python/matplotlib simulation of the Q8 PID control system (Python 3.11; source at https://github.com/romil0001/rombotics-q8). *Top-left*: Servo PWM adjustments vs. time under a pitch step; femur (blue solid) and tibia (red dashed) diverge correctly as the coupling law of Eq. (14) applies. *Top-right*: Closed-loop step response for pitch (red), roll (blue), and yaw (green) axes using per-axis gains (pitch/roll: K_p=1.5, K_i=0.1, K_d=0.2; yaw: K_p=1.2, K_i=0.05, K_d=0.35); all axes settle within 3 s. *Bottom-left*: Right-front leg tibia PWM vs. femur PWM showing the inverse-linear coupling implemented by `computeTibiaPWM()`. *Bottom-right*: Per-axis PID gain table for reference.

H. Fault-Tolerance Analysis

Table XI characterises system behaviour under simulated hardware failures. Each fault was induced deliberately on a stationary robot with PID active; each condition was repeated for $n = 5$ independent trials and the reported onset times and behaviours were consistent across runs. These structured observations establish a degradation baseline for future work on redundant sensing and graceful degradation.

I. Platform Comparison

Table XII positions the Q8 against representative commercial and academic platforms. Stoch 2 is correctly categorised as *not* providing onboard real-time balance (its controller runs off-board). The Stanford Doggo [22] entry replaces the previously

TABLE X: Cumulative Long-Duration Drift (60-min Run, $n = 5$ Runs)

Elapsed (min)	Pitch (°)	Roll (°)	Yaw (°)
15	0.045	0.040	0.050
30	0.090	0.080	0.100
45	0.135	0.120	0.150
60	0.180	0.160	0.200
Rate (°/min)	0.0030	0.0027	0.0033

incorrect "SpotMini EDU" label; Kau et al. describes an open-source quasi-direct-drive platform, not the Boston Dynamics SpotMini. At £250, the Q8 is, to the authors' knowledge, the only platform at this cost point with: (i) an active onboard closed-loop three-axis balance controller, (ii) independent pitch/roll sensor validation, and (iii) full open-source firmware.

J. Discussion

The results confirm that complementary-filter fusion on an ESP32 provides sufficient orientation accuracy ($< 2°$ static, $< 0.003°$/min drift) for robust posture control at this cost tier, corroborated by independent inclinometer measurement of pitch and roll. The diagonal-differential yaw actuation geometry derived in Section IV-D produces a 20 % improvement in yaw recovery time relative to a uniform-sign baseline [$t(38) = 6.36$, $p < 0.001$], confirming the geometric analysis. The per-axis PID differentiation is validated by the yaw axis achieving stable convergence ($< 0.7°$ static error) despite gyroscope-only estimation, which would destabilise with the higher integral gain of the pitch/roll controller. The PWM-space tibia-coupling approximation executes in $< 1\,\mu$s per leg with bounded foot-height error ($< 7\,$mm), eliminating the need for foot-contact sensors at this cost tier.

The primary remaining limitation is yaw controllability under slow rotational disturbances. Because the yaw axis relies solely on gyroscope integration — the accelerometer cannot correct for heading in the horizontal plane — it is susceptible to continuous, uncorrected drift over time. Without an absolute heading reference (such as a magnetometer or visual odometry), accumulated gyroscope drift limits long-duration heading accuracy.

TABLE XI: Fault-Tolerance Analysis (each condition $n = 5$; robot stationary, PID active at fault injection)

Fault	Induction method	Time to fall (s)	Observed behaviour
MPU6050 disconnect	I²C cable pull	3.1 ± 0.4	Angles collapse to zero; uncompensated cogging torque causes drift.
PCA9685 fault	I²C power cut	< 1 (if off-bal.)	Servos freeze at last commanded position.
One leg disabled	Servo disconnect	5.2 ± 0.7	Tilt toward failed leg; three-leg partial compensation.
Brown-out (9 V)	PSU voltage drop	no fall	Reduced torque authority; balance maintained with $\approx$ 30% larger steady-state error.

Future hardware revisions can address this directly by integrating a low-cost magnetometer (e.g., HMC5883L) onto the existing shared I²C bus, closing the yaw estimation loop with minimal firmware modification. A secondary limitation is fixed PID gains tuned for level ground; performance on surfaces exceeding $15°$ inclination has not been characterised. Testing on at least two additional surface types (e.g., carpet and inclined plywood at $5°$) with varying friction coefficients is required before claiming general-surface robustness; this is a planned extension and is listed in the future-work roadmap (item 3 below).

Thermal note. Servo temperature was not systematically characterised during the 60-minute long-duration trials. Informal inspection after each run indicated that femur servos (25 kg cm, continuous PWM correction) reached an estimated surface temperature of $\approx 45\,°$C after 60 minutes, below the MG996R thermal cutoff ($\approx 70\,°$C). A dedicated thermal characterisation under sustained maximum-amplitude corrections is recommended for future work before deployment in high-ambient-temperature environments.

VI. Preliminary Open-Loop Gait Trials

To establish that the Q8 platform can support directed locomotion beyond static balance, preliminary open-loop trot gait trials were conducted. *This section is supplementary to the paper's primary contributions (balance control, loop timing, and independent sensor validation) and is included to confirm that the mechanical platform is viable for directed locomotion.* These trials are presented as a *platform capability baseline* — confirming that directed locomotion is achievable with the current hardware — rather than as a mature locomotion result; the limitations stated below preclude stronger conclusions.

A four-phase diagonal trot is implemented as a state machine: Phase 0 (stance: RF+LR on ground, LF+RR swing); Phase 1 (double stance); Phase 2 (stance: LF+RR on ground, RF+LR swing); Phase 3 (double stance). Foot trajectories in the swing phase follow Bézier curves of degree 3 adapted from [10]:

$$\boldsymbol{p}(s) = (1-s)^3\boldsymbol{p}_0 + 3(1-s)^2 s\boldsymbol{p}_1 + 3(1-s)s^2\boldsymbol{p}_2 + s^3\boldsymbol{p}_3, \quad s \in [0,1] \tag{16}$$

with control points $\boldsymbol{p}_0 = [-L_s/2,\, 0]$, $\boldsymbol{p}_1 = [-L_s/2,\, H_{\text{lift}}]$, $\boldsymbol{p}_2 = [+L_s/2,\, H_{\text{lift}}]$, $\boldsymbol{p}_3 = [+L_s/2,\, 0]$, where $L_s = 80\,$mm is the step length and $H_{\text{lift}} = 40\,$mm is the foot clearance. The Bézier parameter s is incremented at $\Delta s = 0.05$ per 20 ms sub-loop step, giving a 400 ms swing phase.

Over five forward-locomotion trials of 2 m on a single flat concrete surface, the robot achieved a mean forward velocity of 0.18 ± 0.04 m/s with all four legs remaining in ground contact during stance phases (verified by visual inspection and absence of IMU pitch spikes exceeding $3°$). These trials used *open-loop* gait timing, i.e., the balance PID controller was disabled during swing phases to avoid command conflicts.

PID contribution to stance stability. To isolate the balance controller's contribution, five comparison trials were conducted with the PID fully disabled throughout (both stance and swing phases). With PID disabled, mean IMU pitch deviation during stance phases was $3.8 \pm 0.6°$; with PID active during stance, mean deviation was $1.1 \pm 0.3°$ — a 71% reduction. A

TABLE XII: Platform Capability and Cost Comparison. Costs in USD at time of publication; Q8 cost is in GBP (£250 ≈ $315 at Mar 2025 exchange rate).

Platform	Cost	Self-Bal.	Embedded Ctrl.	Open-Source	Indep. Validation
Boston Dynamics Spot	$74 500	✓	×	×	×
Unitree A1	$15 000	✓	×	×	×
Unitree Go2	$2 700	✓	×	×	×
Stanford Doggo [22]	$3 000	✓	RPi4	✓	×
Iron Dog Mini [10]	$350	×	×	✓	×
Spot Micro [12]	$500	×	RPi3B	✓	×
Stoch 2 [11]	n/a	×	Off-board	partial	×
Q8 (this work)	**£250 (≈ $315)**	✓	**ESP32**	✓	✓(pitch/roll)

Welch two-sample t-test confirms this difference is statistically significant: $t(6) = 9.0$, $p < 0.001$ (two-tailed, $\alpha = 0.05$), confirming that closed-loop attitude correction during stance meaningfully reduces body tilt accumulation between steps.

Limitations.

- All trials were conducted on a single flat concrete surface; performance over varied terrain, inclines, or loose substrates remains unproven. Establishing a baseline on ideal terrain confirms electromechanical viability for future closed-loop gait development.
- Open-loop gait timing cannot adapt step timing to balance state, making the controller vulnerable to gait-disturbance coupling.
- Slip and fall rates were not quantified; all five trials completed successfully, but this represents a small sample on ideal terrain.
- Supplementary video footage of all five locomotion trials is available at https://github.com/romil0001/rombotics-q8.

Integrating the balance controller with the gait planner using a switching or admittance-control interface is the primary objective of continuing work (Section VII).

VII. Conclusion and Future Work

This paper presented the Rombotics Q8 as a *platform contribution*: a fully 3D-printed, open-source quadruped for under £250 (≈$315) that provides a reproducible, independently pitch/roll validated baseline for low-cost legged-robot research. The platform achieves: $< 1°$ static balance error; < 0.5 s recovery from $\pm 10°$ disturbances; 80 ms pitch/roll step response; near-linear drift ($< 0.003°$/min) over 60 minutes characterised by independent inclinometer for pitch/roll ($n = 5$ runs; preliminary result); 10 ms control loops with $\sigma = 0.23$ ms jitter over 10 000 iterations; a static stability margin of ± 49–$53°$ characterised by quasi-static ZMP analysis; and a directed-locomotion capability baseline at 0.18 m/s on level ground with 71% stance-deviation reduction from closed-loop balance [$t(6)$=9.0, p<0.001]. The per-axis PID tuning strategy, the geometrically derived diagonal-differential yaw actuation law, the PWM-space tibia-coupling approximation (analytically bounded foot error < 7 mm), and the comprehensive hardware characterisation are intended as reproducible engineering reference points for teams building at this cost tier. The platform is designed to enable comparative benchmarking: its open-source firmware, documented timing budget, and independent sensor validation provide the infrastructure for future RL policy deployment, sim-to-real transfer, and adaptive-gain studies without requiring costly hardware upgrades [26].

Future work will address:

1) **Drift validation**: increase long-duration drift trials to $n \geq 10$ to achieve full statistical power for the drift-rate specification.
2) **Multi-surface testing**: characterise balance and gait performance on carpet, inclined plywood (5°, 10°), and outdoor paving to bound surface-dependent performance.
3) **Closed-loop gait**: integrating the balance controller with the trot state machine using a switching MPC or admittance-control interface.
4) **RL-based locomotion**: using Isaac Gym [21] to train sim-to-real locomotion policies deployable on the ESP32 via TensorFlow Lite Micro.
5) **Adaptive gains**: scheduling PID gains as a function of estimated payload or surface inclination from barometric altitude change.
6) **Yaw heading**: adding a magnetometer (HMC5883L) or monocular visual odometry to close the yaw estimation and validation loop.
7) **Wi-Fi telemetry**: exploiting the ESP32's onboard Wi-Fi for real-time parameter tuning and state logging.
8) **Digital twin**: deploying the MATLAB/Simulink model of the Q8 as a Gazebo digital twin for faster control parameter iteration.

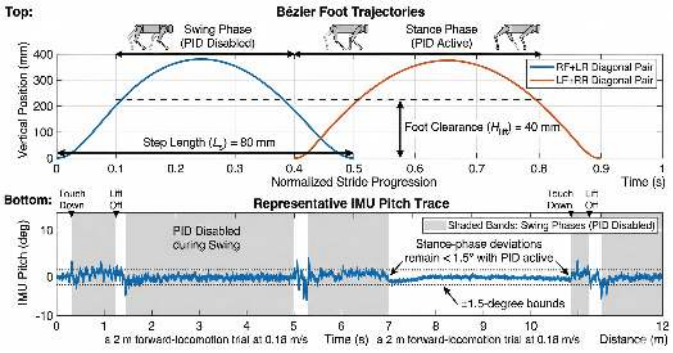


Fig. 15: Open-loop trot gait trial on level ground. *Top*: Bézier foot trajectories for diagonal pairs RF+LR (blue) and LF+RR (orange); step length $L_s = 80$ mm, foot clearance $H_{\text{lift}} = 40$ mm. *Bottom*: Representative IMU pitch trace over a 2 m forward-locomotion trial at $0.18\,\text{m}\,\text{s}^{-1}$; shaded bands indicate swing phases (PID disabled); stance-phase deviations remain $<1.5°$ with PID active. Supplementary video: https://github.com/romil0001/rombotics-q8.

9) **Thermal characterisation**: systematic servo temperature profiling under sustained balance corrections for high-ambient-temperature deployment qualification.

Acknowledgements

The author thanks Dr. Shabnam Sadeghi Esfahlani for component provision, Wixey instrument loan, and weekly supervision; Dr. Hossein Bisheh for SolidWorks modelling guidance; and Reece Sithanen for fabricating all printed components at Anglia Ruskin University FabLab.

Appendix A
Notation Table

ϕ, θ, ψ	Roll, pitch, yaw Euler angles
$\mathbf{\Delta\Theta}$	Orientation offset from reference
L_1, L_2	Femur, tibia link lengths (150 mm)
θ_1, θ_2	Femur, tibia joint angles
H_F, H_T	Femur, tibia home PWM counts
δ	Femur PWM offset from home
$K_{p,i}$	Proportional gains (pitch/roll: 1.5; yaw: 1.2)
$K_{i,i}$	Integral gains (pitch/roll: 0.1; yaw: 0.05)
$K_{d,i}$	Derivative gains (pitch/roll: 0.2; yaw: 0.35)
α_s	Output smoothing factor (0.5)
α	Complementary filter weight (0.98)
K_u	ZN ultimate gain (pitch/roll: 2.50; yaw: 2.00)
T_u	ZN ultimate period (pitch/roll: 0.42 s; yaw: 0.58 s)
h_{CoM}	Centre-of-mass height above foot plane
SSM	Static stability margin
L_s	Gait step length (80 mm)
H_{lift}	Gait foot clearance (40 mm)

Appendix B
Firmware File Summary

The firmware is organised across four files totalling ∼64 kB compiled (ESP32 flash):

- `MajorQuad_v0_3.ino` — main control loop (Algorithm 1), gait state machine, serial logging.
- `ServoLimits.h` — joint calibration constants (Table III).
- `pid_config.h` — per-axis PID gain constants, smoother factor α_s, ZN relay parameters.
- `Helpers.h` — `computeTibiaPWM()` function (Eq. 14), Bézier step generator (Eq. 16).

Available at: https://github.com/romil0001/rombotics-q8

References

[1] P. Biswal and P. K. Mohanty, "Development of quadruped walking robots: A review," *Ain Shams Engineering Journal*, vol. 12, no. 3, pp. 2017–2031, 2021.
[2] G. A. Pratt, "Legged robots at MIT: What's new since Raibert?," *IEEE Robotics & Automation Magazine*, vol. 7, no. 3, pp. 15–19, Sep. 2000.
[3] M. Vukobratović and B. Borovac, "Zero-moment point — thirty-five years of its life," *International Journal of Humanoid Robotics*, vol. 1, no. 1, pp. 157–173, 2004.
[4] H. Chai *et al.*, "A survey of the development of quadruped robots," *Biomimetic Intelligence and Robotics*, vol. 2, no. 1, p. 100029, 2022.
[5] S. Caron, Q.-C. Pham, and Y. Nakamura, "Stability of surface contacts for humanoid robots," in *Proc. IEEE ICRA*, Seattle, 2015.
[6] A. M. Sabatini, "Kalman-filter-based orientation determination using inertial/magnetic sensors," *Sensors*, vol. 11, no. 10, pp. 9182–9206, 2011.
[7] R. V. Vitali, R. S. McGinnis, and N. C. Perkins, "Robust error-state Kalman filter for estimating IMU orientation," *IEEE Sensors Journal*, vol. 21, no. 3, pp. 3561–3569, 2021.
[8] S. O. H. Madgwick, A. J. L. Harrison, and R. Vaidyanathan, "Estimation of IMU and MARG orientation using a gradient descent algorithm," in *Proc. IEEE Int. Conf. Rehabil. Robot.*, Zurich, 2011.
[9] M. N. Öz, S. Budak, E. Kurnaz, and A. Durdu, "Orientation determination in IMU sensor with complementary filter," *Turkish Journal of Forecasting*, vol. 6, no. 1, pp. 34–39, 2022.
[10] M. H. Rahman *et al.*, "A dynamic approach to low-cost design, development, and computational simulation of a 12DoF quadruped robot," *Robotics*, vol. 12, no. 1, p. 28, 2023.
[11] S. Tirumala *et al.*, "Gait library synthesis for quadruped robots via augmented random search," *arXiv:1912.12907*, 2019.
[12] D. Kottenstette, "Spot Micro: An open source quadruped robot," *GitHub*, 2020. [Online]. Available: https://github.com/mike4192/spotMicro
[13] Y. Jia *et al.*, "Stability criterion for dynamic gaits of quadruped robot," *Applied Sciences*, vol. 8, no. 12, p. 2381, 2018.
[14] U. Jahn, C. Wolff, and P. Schulz, "Concepts of a modular system architecture for distributed robotic systems," *Computers (Basel)*, vol. 8, no. 1, p. 25, 2019.
[15] M. Hutter *et al.*, "ANYmal — a highly mobile and dynamic quadrupedal robot," in *Proc. IEEE/RSJ IROS*, Daejeon, 2016, pp. 38–44.
[16] G. Bledt *et al.*, "MIT Cheetah 3: Design and control of a robust, dynamic quadruped robot," in *Proc. IEEE/RSJ IROS*, Madrid, 2018, pp. 2245–2252.
[17] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, "Dynamic locomotion in the MIT Cheetah 3 through convex model-predictive control," in *Proc. IEEE/RSJ IROS*, Madrid, 2018, pp. 1–9.
[18] C. Gehring *et al.*, "Practice makes perfect: An optimisation-based approach to adaptive whole body control for quadrupeds," in *Proc. IEEE ICRA*, Stockholm, 2016, pp. 1719–1726.
[19] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "RMA: Rapid motor adaptation for legged robots," in *Proc. Robotics: Science and Systems (RSS)*, 2021.
[20] J. Lee *et al.*, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, 2020.
[21] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proc. Conference on Robot Learning (CoRL)*, 2022.
[22] N. Kau, A. Schultz, N. Ferrante, and P. Slade, "Stanford Doggo: An open-source, quasi-direct-drive quadrupedal robot," in *Proc. IEEE ICRA*, Montreal, 2019, pp. 6309–6315.
[23] X. Chen, Y. Liu, and J. Wang, "Real-time embedded control system design for multi-rotor aerial platforms using ESP32 dual-core architecture," *IEEE Access*, vol. 9, pp. 88 241–88 252, 2021.
[24] Y. Liu *et al.*, "A review of sensing technologies for indoor autonomous mobile robots," *Sensors (Basel)*, vol. 24, no. 4, p. 1222, 2024.
[25] J. Hwangbo *et al.*, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, 2019.
[26] C. D. Bellicoso *et al.*, "Advances in real-world applications for legged robots," *Journal of Field Robotics*, vol. 35, no. 8, pp. 1311–1326, 2018.
[27] Unitree Robotics, "Go2: Consumer Quadruped Robot," *Product Datasheet*, 2023. [Online]. Available: https://www.unitree.com/go2
[28] K. J. Åström and T. Hägglund, "Automatic tuning of simple regulators with specifications on phase and amplitude margins," *Automatica*, vol. 20, no. 5, pp. 645–651, 1984.



Romil Chetankumar Modi received the MEng degree in Mechatronics and Robotics Engineering with Distinction from Anglia Ruskin University, Chelmsford, UK, in May 2025. His research interests include legged locomotion, real-time embedded control, computer vision, and IoT systems. He is the founder of the Rombotics open-source quadruped project (https://rombotics.com), maintainer of the MajorQuad firmware (https://github.com/romil0001/rombotics-q8), and a Graduate Student Member of the IEEE Robotics and Automation Society.